

DIGIT MEMORY

*A study in retrieval as classification.
Built three ways. No model trained.*

98.4%

HELD-OUT ACCURACY

1-NN on a stratified 75/25 split of the sklearn digits dataset.

CONTENTS

01	Cover
02	The hypothesis
03	Architecture
04	Numbers
05	Dataset
06	Robustness
07	Three implementations
08	Colophon

Hypothesis

01

The fastest model is the one you don't need. Most ML projects start with what model to use. This one started with a different question.

What if you just stored the data, organized it well, and looked things up? For a dataset of 1797 small samples in 64 dimensions, the answer turns out to be: that's enough.

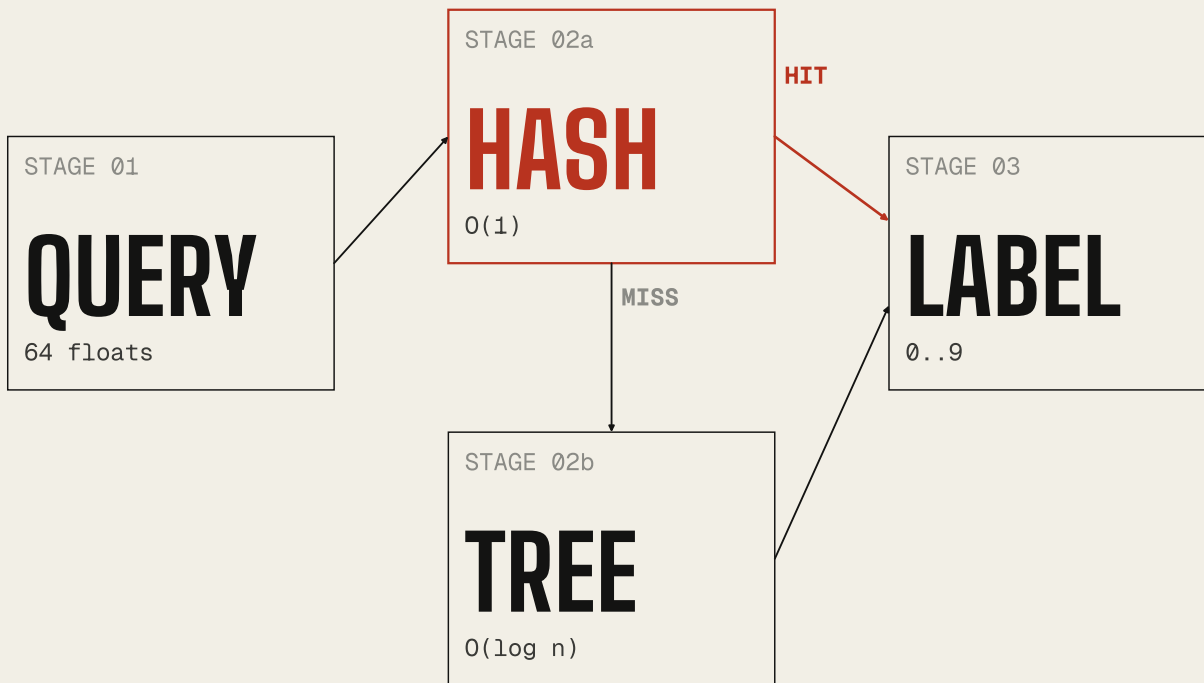
Two query patterns drive the design. An exact match — the input is byte-identical to something seen — wants a hash table. $O(1)$ lookup. There is no smarter answer.

An approximate match — the input is close but not identical — wants a spatial index. Hashing breaks: change one bit and the hash changes completely. We need a structure that knows about geometric closeness.

The combined system tries the cheap lookup first and falls back to the expensive one only when needed. That is the entire idea.

Two-tier lookup

FIG.01 HASH FIRST · TREE ON MISS



Most production lookup systems quietly do this — caches in front of expensive indexes, content-addressed stores in front of full-text search. The same pattern applies cleanly to classification when the data fits in memory.

Numbers

HELD-OUT ACCURACY

98.4%

stratified 75/25 split · 1-NN · no training

EXACT LOOKUP

3_{us}

Python dict, bytes(features) -> label

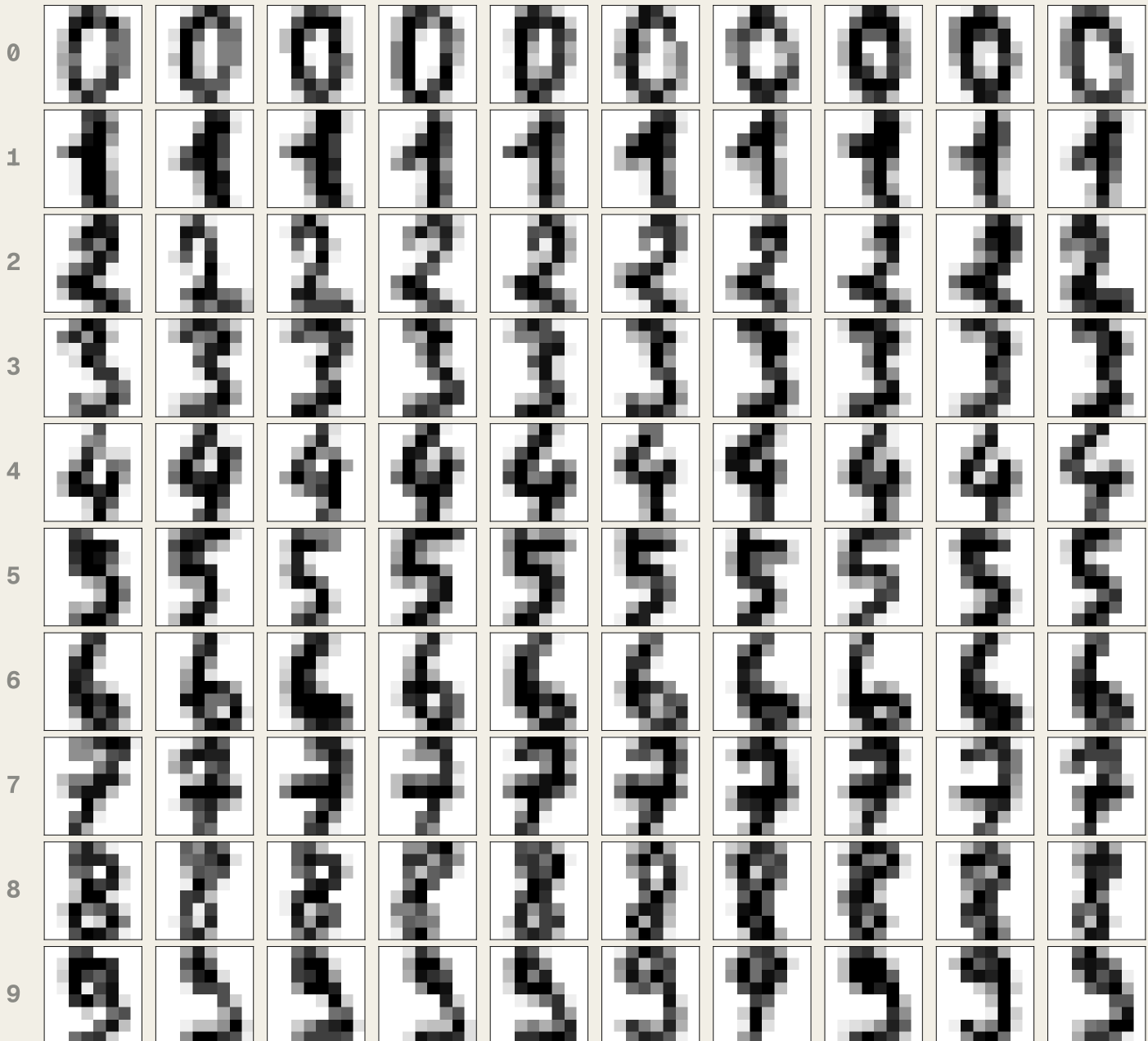
NEAREST NEIGHBOR

50_{us}

BallTree query, k = 1, 1797 samples

Dataset

Ten samples per class. The variation across rows is the noise the memory has to tolerate.

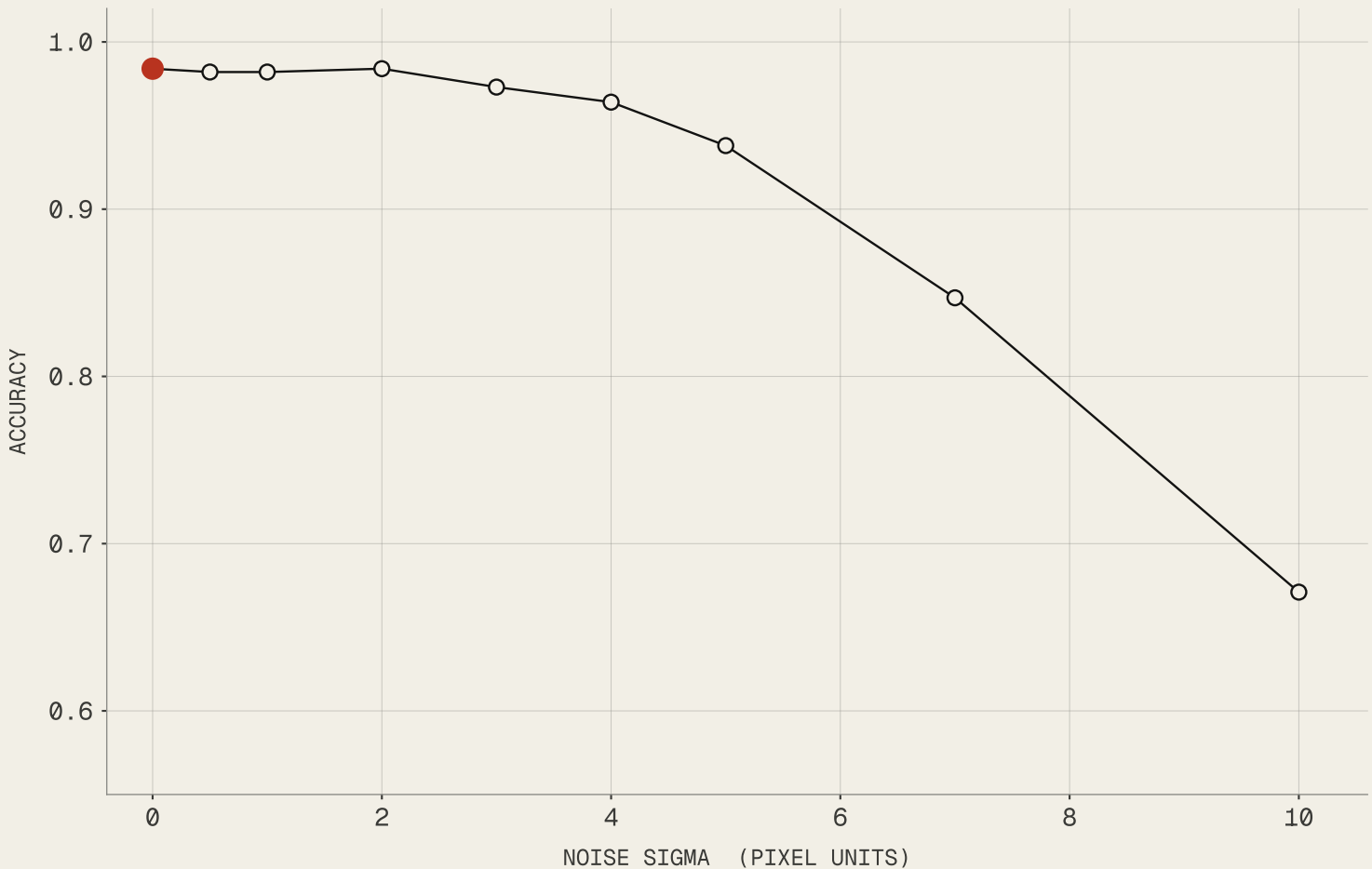


1,797 SAMPLES · 8x8 GRAYSCALE · 16 INTENSITY STEPS

Classes balanced: ~180 examples per digit.

Robustness

Accuracy on the held-out test set with injected Gaussian noise.



sigma = 0 -> 98.4% **sigma = 5 -> 93.8%** **sigma = 10 -> 67.1%**

Smooth degradation, no cliff. That's what robust retrieval looks like.

Three ways

Writing the same system three times is the closest I have found to actually understanding it.

01 PYTHON

[digits_memory.py](#)

Production-style. SQLite persistence. sklearn KDTree.

```
import sqlite3, numpy, sklearn.neighbors
```

Use when: you want the cleanest summary.

02 JUPYTER

[digits_study_guide.ipynb](#)

43 cells. Pandas DataFrames. Three NN backends benchmarked.

```
%timeit · pd.DataFrame · plt.imshow
```

Use when: you want to learn the project hands-on.

03 C

[digits_memory.c](#)

Zero dependencies. FNV-1a open-addressed hash. Brute-force NN.

```
gcc -O2 -o digits_memory digits_memory.c -lm
```

Use when: you want to understand every byte.

Colophon

DESIGN

PHILOSOPHY	Computational Quiet
DISPLAY	Big Shoulders Bold
BODY	Instrument Sans
SERIF ACCENT	Instrument Serif Italic
MONO	Geist Mono
INK	#131312
PAPER	#F2EFE6
ACCENT	#B8331F

ENGINEERING

DATASET	sklearn.datasets.load_digits · 1797 x 64
EXACT MATCH	Python dict, bytes(features) key
NN PYTHON	sklearn BallTree / KDTree
NN C	FNV-1a hash + brute force
PERSISTENCE	SQLite (.db) + CSV export
ACCURACY	98.4% held-out, 1-NN

ALL CODE OPEN SOURCE · MIT

github.com/nimb-ou/ML_projects

Nimit Jain · 2026